

2 Ricerca in Spazi di Stati

2.1 Risoluzione di Problemi mediante Ricerca

La risoluzione di diversi problemi spesso consiste nella *ricerca* in uno specifico modello del mondo. Si consideri un caso semplice, quello di un sistema che, dato un *obiettivo* da raggiungere, ragiona su un *modello* del mondo fatto di *stati*, in *assenza di incertezza*. A tal fine, si supponga di adottare una rappresentazione *piatta*, i.e. non gerarchica, ovvero relativa a un singolo livello in una gerarchia.

Astrazione del problema: in un *grafo orientato*, ricerca di un *percorso* che vada da un nodo di partenza a uno dei nodi-obiettivo (*goal*). Molti problemi possono essere mappati su quest'astrazione per la quale sono disponibili diversi algoritmi di ricerca.

Esempio 2.1 (navigatore) L'obiettivo è la ricerca del *miglior percorso* da un luogo a un altro: il più *corto* / minima distanza; quello di minimo *costo* (ad esempio in termini di carburante, tempo, ecc.); il più *veloce*; il più *attrattivo* (ad esempio con più POI, *point-of-interest*).

Ogni *stato* include informazioni su: localizzazione, direzione, velocità, mezzo di trasporto utilizzato, ...

Una strategia di ricerca comune a molti problemi in AI è la seguente: il sistema lavora (computazione) su una sua *rappresentazione interna* (diversamente dalla ricerca operata da robot che *agisce* direttamente sul mondo fisico e dalla ricerca sul web che mira al ritrovamento di informazioni). Il sistema riceve solo una descrizione di *cosa* rappresenti una soluzione; *come* ottenerla sarà il compito di un algoritmo di ricerca. Si osservi che molto spesso tali problemi e i relativi algoritmi risolutivi sono **NP-completi**, anche nei casi in cui sia disponibile un modo *efficiente* per riconoscere le soluzioni.

Tali problemi risultano *difficili* anche per agenti umani. Spesso non esistono soluzioni ottimali generali per cui ci si accontenta di soluzioni *soddisfacenti* in mancanza d'una *struttura* mappabile su caratteristiche del mondo fisico. Ad esempio nella *crittografia* si richiede che i problemi risultino estremamente difficili anche per macchine, date le comuni limitazioni di tempo e spazio. Come si vedrà più avanti, converrà sfruttare conoscenza aggiuntiva (*euristica*), se disponibile.

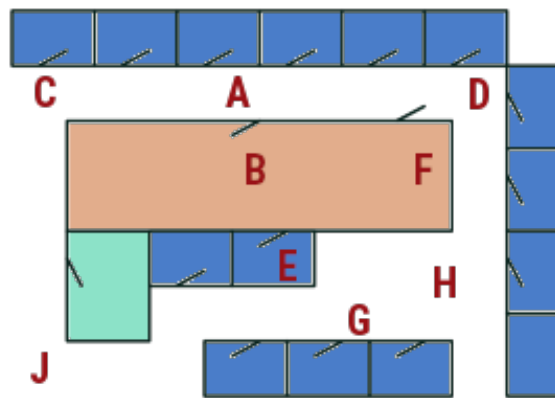


Figura 2.1: Problema di ricerca [PM23]: posizioni in un piano di un edificio

2.2 Spazi di Stati

Formuliamo il problema in termini di uno **spazio di stati**. Uno **stato** deve contenere tutta l'informazione necessaria a predire gli effetti di un'azione (ad esempio una mossa in un gioco) e a verificare se esso soddisfa l'obiettivo (*stato obiettivo*). Una **soluzione** sarà una sequenza di azioni che portano dallo stato corrente a uno stato obiettivo (fine-percorso $\rightarrow$ soluzione).

Le *assunzioni* comuni che si fanno sono la conoscenza *perfetta* dello spazio, ossia che lo stato corrente sia noto e il mondo sia *completamente osservabile*. Essenziale è anche l'esistenza di stati-obiettivo, ossia *goal* da raggiungere *riconoscibili* (facilmente). Si assume, inoltre, che a disposizione del sistema/agente sia previsto un insieme di *azioni* dagli effetti *deterministici* noti.

Un **problema di ricerca** in uno *spazio di stati* comprende:

- lo **spazio di stati**: insieme di *stati*;
- uno **stato** (o un sotto-insieme di stati) distinto di **partenza**;
- un insieme di **azioni** possibili per ciascuno stato;
- la **funzione d'azione** (detta anche o *modello di transizione* in [RN20]): data una coppia stato-azione (s, a) , calcola il nuovo stato in cui si transita per effetto dell'azione;
- un **obiettivo**, funzione booleana $goal(s)$ che ha valore *vero* sse s è uno **stato obiettivo**; in alternativa si possono più semplicemente elencare gli stati-obiettivo, se in numero finito;
- un **criterio di qualità** per soluzioni **accettabili**: una **soluzione ottimale** massimizza tale criterio, ad esempio si ammette *qualsiasi* sequenza di azioni che porti a uno stato-obiettivo; ad esempio associando *costi* alle diverse azioni

da cui discende un criterio del *minimo costo totale* delle azioni; si noti che, a volte, sono soddisfacenti anche soluzioni sub-ottimali, ad esempio quelle con un costo aggiuntivo del +10% rispetto a quello di una soluzione ottimale.

Esempio 2.2 (robot-consegne) Un robot che debba effettuare consegne in un edificio dovrà essere in grado risolvere *problemi di ricerca* di percorsi da un luogo ad un altro in un dato piano, come quello illustrato in Fig. 2.1. Occorrerà definire:

- gli *stati*: posizioni distinte (stanze/esterni-stanza);
- le *azioni*: spostamenti da un luogo a un altro nelle vicinanze;
- il *problema* specifico: portarsi da A a G (unico stato-obiettivo);
- la *soluzione*: sequenza di spostamenti.

Possibili *estensioni* dei problemi di ricerca riguardano:

- la possibilità di sfruttare una *struttura interna* agli stati: ad esempio nei videogame;
- il caso di stati *non* completamente *osservabili*: ad esempio il robot-consegne non conosce la posizione iniziale delle consegne, oppure un sistema di tutoring può non conoscere bene le attitudini di uno studente;
- l'aggiunta di azioni *stocastiche*: ad esempio possibilità di commettere errori (robot-consegne) oppure mancato apprendimento da parte dello studente di un argomento;
- la possibilità di definire, in vece degli stati finali, *preferenze aggiuntive* complesse, in termini di *ricompense* o *punizioni*.

Esempio 2.3 (videogame) Si consideri lo spazio in Fig. 2.2. In tal caso:

- *modello*: griglia nella quale ci si può muovere di una casella nelle 4 direzioni se non si è bloccati da un muro (caselle scure);
- *obiettivo*: raccogliere 4 monete $M_1, \dots, M_4$, ognuna in una posizione iniziale nota, ad esempio C_3 in (5,7);
- *costo*: un'unità di energia per ogni passo: nessuna mossa è possibile senza energia; ricarica (+16 unità) presso una casella *ricarica*, ad esempio in (4,9);
- *stato*: dato da una posizione x, y , dalle unità di carburante u possedute e da 4 flag c_i che indicano il possesso della relativa moneta: $(x, y, u, m_1, m_2, m_3, m_4)$;

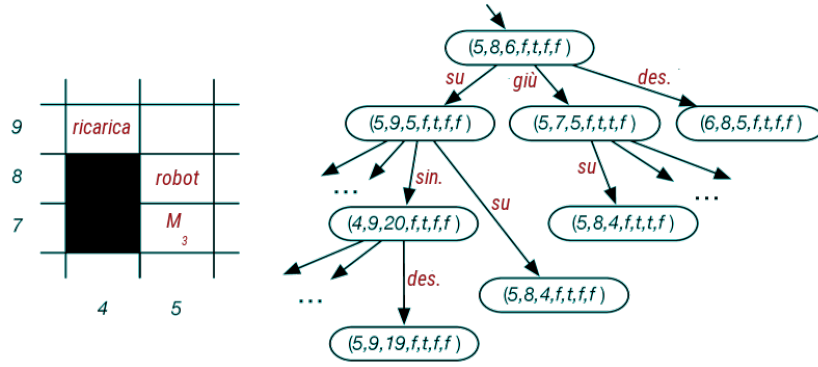


Figura 2.2: Problema da [PM23]

- possibili stato-obiettivo $(5,7,?,t,t,t,t)$ (dove ? indica un quantitativo qualsiasi di carburante).

2.3 Ricerca su Grafo

Per risolvere il problema, si definisce lo spazio di ricerca e si applica un algoritmo di ricerca. Come anticipato, molti compiti possono essere ricondotti alla *ricerca* di percorsi in un grafo. Un modello *astratto* della soluzione sarà *indipendente* dal particolare dominio applicativo di interesse. Dato un *grafo* (*orientato*) fatto di nodi connessi da archi, si dovrà trovare un *percorso* (di più archi) da un nodo di partenza a uno obiettivo. Ci possono essere più maniere per rappresentare uno stesso problema.

Un **grafo orientato esplicito** (o anche *implicito*, se è disponibile una procedura per generare nodi e archi) consiste di:

- un insieme di *nodi* N , anche non finito;
- un insieme di *archi* A , ossia di coppie *ordinate* di nodi, anche non infinito;
 - l'arco $\langle n_1, n_2 \rangle$ si dirà **uscente** da n_1 ed **entrante** in n_2 ; esso può anche essere etichettato con l'azione che porta da n_1 a n_2 ;
 - si dirà che il nodo n_2 è un **vicino** del nodo n_1 sse $\exists \langle n_1, n_2 \rangle \in A$ (si noti che la relazione *non* è necessariamente *simmetrica*);
- un **percorso** (o *cammino*) dal nodo s al nodo g , denotato da $\langle n_0, n_1, \dots, n_k \rangle$, è una *sequenza* di nodi tale che $s = n_0$, $g = n_k$ e $\forall i = 1, \dots, k: \langle n_{i-1}, n_i \rangle \in A$; in alternativa, si può indicare una *sequenza di archi* $\langle n_0, n_1 \rangle, \langle n_1, n_2 \rangle, \dots, \langle n_{k-1}, n_k \rangle$ o anche *sequenza di etichette* su tali archi: $\langle n_0, \dots, n_i \rangle$ parte iniziale di $\langle n_0, n_1, \dots, n_k \rangle$, con $i \leq k$.

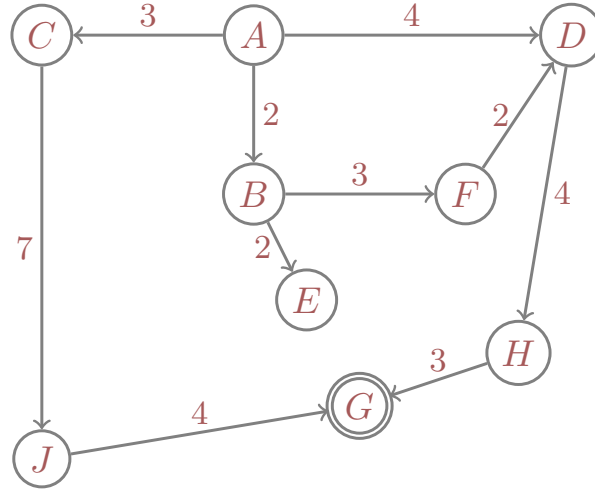


Figura 2.3: Grafo che rappresenta schematicamente il problema di ricerca discusso in precedenza (Fig. 2.1)

Nel grafo esiste un insieme dei *nodi-obiettivo* (*goal*) che si possono identificare anche tramite il predicato $goal(\cdot)$, funzione booleana definita sui nodi. Una **soluzione** è un percorso dal nodo di partenza a uno obiettivo.

In alcuni casi è associato un **costo** a ogni arco: dato $\langle n_i, n_j \rangle$

$$cost(\langle n_i, n_j \rangle) \in [0, +\infty[$$

estendibile anche ai percorsi: dato $p = \langle n_0, n_1, \dots, n_k \rangle$

$$cost(p) = \sum_{i=1}^k cost(\langle n_{i-1}, n_i \rangle) = cost(\langle n_0, n_1 \rangle) + \dots + cost(\langle n_{k-1}, n_k \rangle)$$

Una soluzione **ottimale** p avrà costo minimo: $\nexists p'$ soluzione con $cost(p') < cost(p)$.

Esempio 2.4 (Robot consegne, cont.) Ricerca di un percorso da A a G nel mondo rappresentato in Fig. 2.3:

- $N = \{A, B, C, D, E, F, G, H, J\}$
- $A = \{\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle, \langle B, E \rangle, \langle B, F \rangle, \langle C, J \rangle, \dots\}$ dove: E non ha vicini; C ha come vicino solo J ; A ha i vicini B , C e D ;
- vi sono 3 percorsi da A a G : $\langle A, D, H, G \rangle$, $\langle A, C, J, G \rangle$, $\langle A, B, F, D, H, G \rangle$; se A fosse nodo di partenza e G il nodo-obiettivo, ognuno di tali percorsi sarebbe una soluzione.

Esercizio – Trovare una soluzione *ottimale* per il problema precedente.

Un **ciclo** è un percorso non vuoto in cui primo e ultimo nodo coincidono: $\langle n_0, n_1, \dots, n_k \rangle$ tale che $k > 0$ e $n_0 = n_k$.

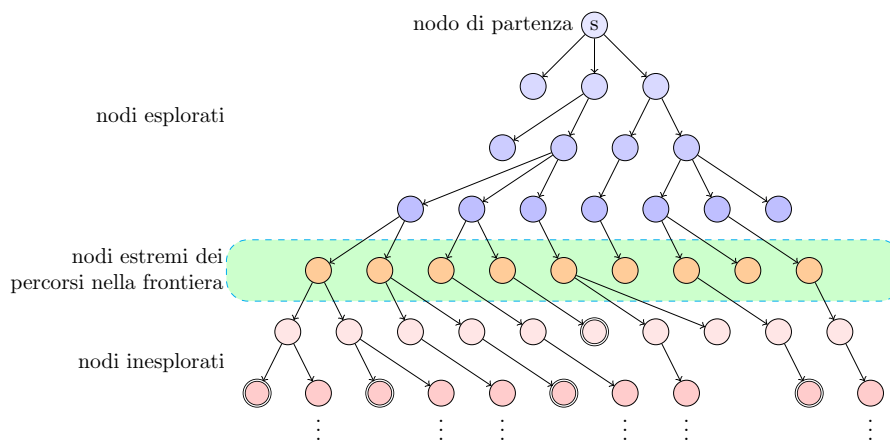


Figura 2.4: Ricerca generica usando una frontiera

Un **grafo aciclico orientato** (*directed acyclic graph*, DAG) è un grafo orientato senza cicli (come quello nella figura precedente).

Un **albero** è un DAG con un solo nodo, la **radice**, senza archi entranti, mentre tutti gli altri ne hanno esattamente uno. I nodi senza archi uscenti sono le sue **foglie**. Si definiscono relazioni fra nodi, come ad esempio genitore-figlio, fratello-sorella, antenato, ecc..., prese in prestito dalla metafora dell'*albero-genealogico*.

Esempio 2.5 Considerato il grafo in Fig. 2.3: per **A**, i fattori di ramificazione entrante e uscente sono pari a 0 e 3, rispettivamente; per **B**, sono 1 e 2; per **D**, 2 e 1.

2.4 Algoritmo di Ricerca Generico

Si tratta di un algoritmo indipendente da strategia di ricerca e/o dal grafo. Dato un grafo, si esplorano *incrementalmente* percorsi dai nodi di partenza verso nodi-obiettivo.

Si usa una struttura dati per rappresentare la **frontiera** (*fringe*) dei percorsi già esplorati, i.e. segmenti iniziali di percorsi da completare fino a un nodo-obiettivo.

Inizialmente la frontiera contiene percorsi costituiti dai soli *nodi di partenza*. Successivamente si effettua l'*espansione* dei percorsi nella frontiera verso nodi inesplorati, fino a incontrare un nodo-obiettivo:

- si seleziona un percorso (rimuovendolo dalla frontiera);
- si estende il percorso con ogni arco uscente dall'ultimo nodo;
- si aggiungono alla frontiera i percorsi ottenuti.

Algoritmo 2.1: Algoritmo di ricerca generico

```

1 procedure Search( $G, s, goal$ )
2
3 Input
4    $G$ : grafo con insiemi di nodi  $N$  e di archi  $A$ 
5    $s$ : nodo di partenza
6    $goal$ : funzione booleana sui nodi
7 Output
8   percorso da  $s$  a un nodo per il quale  $goal$  sia vera
9   oppure  $\perp$  quando non ci sono percorsi / soluzioni
10 Local
11    $frontiera$ : insieme di percorsi
12
13  $frontiera \leftarrow \{s\}$ 
14 while  $frontiera \neq \emptyset$  do
15   selezionare (e rimuovere)  $\langle n_0, \dots, n_k \rangle$  da  $frontiera$ 
16   if  $goal(n_k)$  then
17     return  $\langle n_0, \dots, n_k \rangle$ 
18    $frontiera \leftarrow frontiera \cup \{\langle n_0, \dots, n_k, n \rangle : \langle n_k, n \rangle \in A\}$ 
19 return  $\perp$ 

```

La selezione *non è deterministica*¹, ha solo impatto sull'efficienza quindi una data strategia di selezione determina il percorso da scegliere. Il `return` annidato può essere interpretato come *temporaneo*, continuando si trovano strade alternative; $\perp$ indica che non vi sono (altre) soluzioni; la condizione $goal(s_k)$ viene testata *dopo* la selezione dalla frontiera e non all'aggiunta del nuovo nodo:

1. a volte esiste un *arco* verso un nodo-obiettivo ma *di costo elevato*: non sempre conviene terminare restituendo subito il percorso trovato potrebbe esserci un percorso di costo inferiore; ciò può essere importante per la minimizzazione di tale costo;
2. va tenuto conto del fatto che lo stesso test $goal()$ può essere *costoso*; un percorso che termini con un nodo che non sia un nodo-obiettivo e che non abbia vicini andrebbe rimosso.

2.5 Strategie di Ricerca non Informate

Il problema specifica il grafo e l'obiettivo mentre la **strategia di ricerca** determina il percorso da selezionare dalla frontiera. Le **strategie non informate** non prendono in considerazione la posizione dell'obiettivo. Se si considera un costo unitario per ogni arco, possibili strategie sono la ricerca *in ampiezza*, quella *in*

¹cfr. specchio **Non-deterministic Choice** in [PM23].

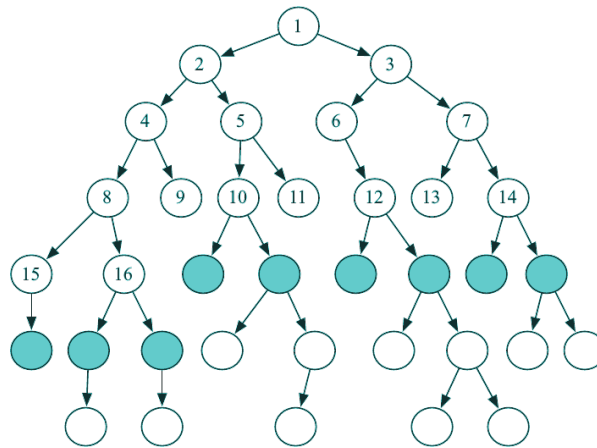


Figura 2.5: BFS: i nodi sono scuri terminali di percorsi della frontiera dopo i primi 16 passi [PM23]

profondità o l'*approfondimento iterativo*. Se è disponibile una funzione di costo, allora si usano strategie dai *costi minimi*.

2.5.1 Ricerca in Ampiezza

Nella **ricerca in ampiezza** (**BREADTH-FIRST SEARCH**, BFS) la frontiera è implementata con una *coda*, ossia una struttura FIFO (first-in, first-out). Si seleziona il *primo* percorso aggiunto. I percorsi sono generati nell'ordine del numero di archi contenuti. A ogni passo, si seleziona uno dei percorsi più corti.

Un esempio albero di ricerca su un dato grafo ottenuto seguendo l'ordine di visita della BFS è illustrato in Figura 2.5.

Esempio 2.6 Dal grafo in Fig. 2.3, ignorando i costi, si ottiene il grafo in Fig. 2.6 con **A** nodo di *partenza* e **G** unico nodo *obiettivo*. Con la BFS:

- frontiera iniziale: $[\langle A \rangle]$;
- estendendo $\langle A \rangle$ con i suoi vicini si ottiene:
 $[\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle]$
 nodi a un arco di distanza da **A**;
- espandendo questi percorsi, nell'ordine si ha:
 $[\langle A, B, E \rangle, \langle A, B, F \rangle, \langle A, C, J \rangle, \langle A, D, H \rangle]$
 percorsi da **A** di lunghezza 2;
- dopo l'espansione dei percorsi del passo precedente la frontiera diventa:
 $[\langle A, B, F, D \rangle, \langle A, C, J, G \rangle, \langle A, D, H, G \rangle]$;
- selezionando, ad esempio, $\langle A, C, J, G \rangle$ questo percorso viene restituito come soluzione.

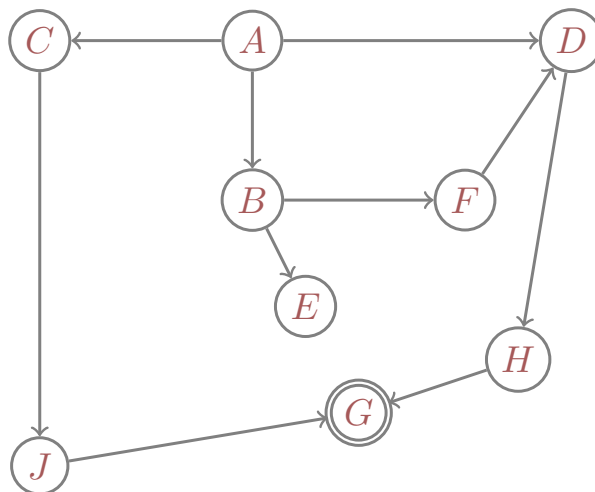


Figura 2.6: Grafo della Fig. 2.3 ma senza informazioni sul costo

NB A ogni passo, nella frontiera vi sono percorsi con approssimativamente lo stesso numero di archi (differenza di un arco).

C'è la *garanzia* di ritrovamento della soluzione quando esiste: una con il minimo numero di archi.

La BFS risulta *utile* quando:

- non si hanno problemi di spazio;
- si cerca una soluzione con numero di archi *minimale*.

La BFS è *poco utile* quando:

- tutte le soluzioni sono associate a percorsi lunghi;
- è disponibile conoscenza euristica;
- il grafo viene generato dinamicamente (a causa della complessità in spazio).

2.5.2 Ricerca in Profondità

Nella **ricerca in profondità** (**DEPTH-FIRST SEARCH**, DFS) la frontiera è implementata con una *pila* (struttura LIFO): gli elementi vengono aggiunti uno alla volta e quello selezionato e prelevato sarà l'ultimo aggiunto.

Partendo dalla radice, i nodi sono considerati come ordinati da sinistra a destra: il vicino più a sinistra sarà aggiunto in cima *per ultimo*. L'ordine di espansione non dipende dalla posizione dei nodi-obiettivo.

Come in precedenza, si osservi in Fig. 2.7 l'albero di ricerca ottenuto seguendo l'ordine della DFS.

Con una pila, la *ricerca* procede *in profondità*:

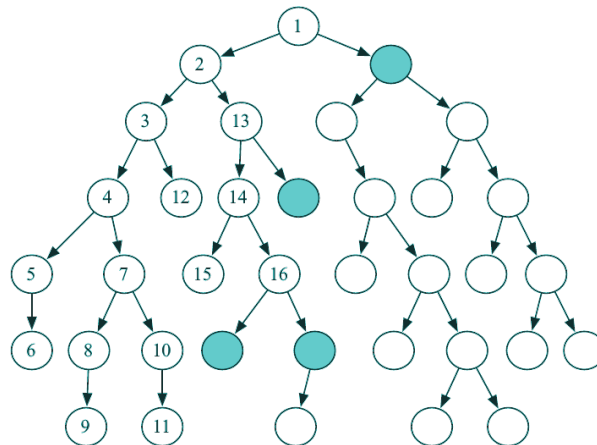


Figura 2.7: DFS: i nodi scuri sono terminali per i percorsi alla frontiera dopo i primi 16 passi: primi sei nello stesso percorso; il nodo 6 non ha vicini quindi il successivo sarà figlio dell’antenato più in basso con figli da espandere [PM23]

- *completamento* di un *singolo percorso* prima di provare alternative;
- questo comporta il **backtracking**: si seleziona una prima alternativa per ogni nodo, *tornando indietro* all'opzione successiva solo dopo aver tentato tutti i completamenti;
- l'*ordine di aggiunta* dei vicini alla frontiera/pila non è specificato: questo impatta sull'efficienza e può essere *statico*, ossia prefissato, ovvero *dinamico*, dipendendo dall'obiettivo;
- per una procedura DFS alternativa (ricorsiva) si veda la Fig. 3.9 in [PM23].

Nota Un grafo con possibili percorsi *infiniti* può portare l'algoritmo a *divergere*; ad esempio se contiene cicli oppure se $|N| = \infty$ (con la generazione dinamica dei nodi).

Esempio 2.7 DFS sul grafo in Fig. 2.6. La pila (frontiera) come lista con il primo percorso in *cima*. Si parte da $[A]$. Le frontiere successive sono:

- $[\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, E \rangle, \langle A, B, F \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F, D \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F, D, H \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F, D, H, G \rangle, \langle A, C \rangle, \langle A, D \rangle]$

Il percorso in cima viene restituito come *soluzione*.

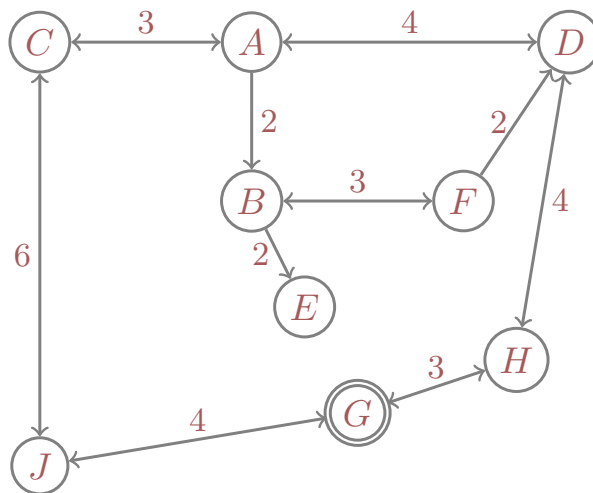


Figura 2.8: Grafo per il quale DFS non termina

Esempio 2.8 Dato il nuovo grafo in Fig. 2.8, sono possibili diversi percorsi infiniti quindi la DFS può divergere trascurando sistematicamente le possibili alternative. Frontiere per le prime iterazioni:

- $\langle A \rangle$
- $\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle$
- $\langle A, B, E \rangle, \langle A, B, F \rangle, \langle A, C \rangle, \langle A, D \rangle$
- $\langle A, B, F \rangle, \langle A, C \rangle, \langle A, D \rangle$
- $\langle A, B, F, B \rangle, \langle A, B, F, D \rangle, \langle A, C \rangle, \langle A, D \rangle$
- $\langle A, B, F, B, E \rangle, \langle A, B, F, B, F \rangle, \langle A, B, F, D \rangle, \langle A, C \rangle, \langle A, D \rangle$
- $\langle A, B, F, B, F \rangle, \langle A, B, F, D \rangle, \langle A, C \rangle, \langle A, D \rangle$... DFS non termina.

Si può migliorare la ricerca *evitando* i percorsi con *cicli* (cfr. potatura).

La DFS è *appropriata* in caso di:

- limitazioni di spazio;
- presenza di molteplici soluzioni, anche se costituite da percorsi lunghi;
- ideale quando *tutti* portano a una soluzione;
- oppure se l'ordine di aggiunta dei vicini può essere variato in modo da trovare una soluzione al *primo tentativo* (senza backtracking);

la DFS risulta *inefficiente* quando:

- sono possibili percorsi infiniti in caso di grafo infinito o contenente cicli;
- pur esistendo soluzioni alternative poco profonde (più a destra), la ricerca si attarda su percorsi più lunghi.

2.5.3 Approfondimento Iterativo

L'obiettivo dell'algoritmo **ITERATIVE DEEPENING** della DFS è combinare l'efficienza in spazio di DFS con l'ottimalità di BFS [RN20, PM23]. L'idea è quella di non memorizzare ma ricalcolare gli elementi della frontiera di BFS usando DFS che usa meno spazio.

DFS effettua una ricerca limitata fino a una data *profondità* come per le iterazioni di BFS. In caso di fallimento si eliminano i percorsi via via calcolati ripartendo, se necessario, dopo aver aumentato il limite. Si parte cercando fino a profondità 1, poi si costruiscono i cammini fino a lunghezza 2, quindi quelli di lunghezza 3, ecc... Se esiste, una soluzione verrà trovata, esplorando percorsi in tale ordine: uno con il minimo numero di archi sarà individuato per primo.

Il *fallimento* della ricerca in profondità limitata può essere:

- *innaturale*: per raggiungimento del limite di profondità e in tal caso la ricerca riparte, dopo aver incrementato il limite;
- *naturale*: quando l'intero spazio di ricerca è esaurito non esiste soluzione a nessun livello di profondità.

Algoritmo 2.2: ITERATIVE DEEPENING

```

1 procedure ID_search( $G, s, goal$ )
2   Input
3      $G$ : grafo con insiemi di nodi  $N$  e di archi  $A$ 
4      $s$ : nodo di partenza
5      $goal$ : funzione booleana sugli stati
6   Output
7     cammino da  $s$  a un nodo per il quale  $goal$  sia vero
8     ovvero  $\perp$  altrimenti
9   Local
10     $raggiunta\_max\_profondita$ : boolean
11     $limite\_profondita$ : integer
12
13    procedure Ricerca_a_profondita_limitata( $\langle n_0, \dots, n_k \rangle, b$ )
14      Input
15         $\langle n_0, \dots, n_k \rangle$ : percorso
16         $b$ : integer  $\{b \geq 0\}$ 
17      Output
18        percorso fino a nodo-obiettivo di lunghezza  $k + b$ 

```

```

19   if  $b > 0$  then
20       for each  $\langle n_k, n \rangle \in A$  do
21            $risultato \leftarrow$  Ricerca_a_profondita_limitata( $\langle n_0, \dots, n_k, n \rangle, b-1$ )
22           if  $risultato \neq \perp$  then
23               return  $risultato$ 
24           else if  $goal(n_k)$  then
25               return  $\langle n_0, \dots, n_k \rangle$ 
26       else if  $n_k$  ha vicini then {fallimento innaturale}
27            $raggiunta\_max\_profondita \leftarrow$  true
28   return  $\perp$ 
29
30  $limite\_profondita \leftarrow 0$ 
31 repeat
32      $raggiunta\_max\_profondita \leftarrow$  false
33      $risultato \leftarrow$  Ricerca_a_profondita_limitata( $\langle s \rangle, limite\_profondita$ )
34     if  $risultato \neq \perp$  then
35         return  $risultato$ 
36      $limite\_profondita \leftarrow limite\_profondita + 1$ 
37 until not  $raggiunta\_max\_profondita$ 
38 return  $\perp$  {fallimento naturale}

```

La procedura `Ricerca_a_profondita_limitata` implementa una DFS (ricorsiva) a profondità limitata: trova percorsi di lunghezza $k+b$, dove k lunghezza del percorso e $b \geq 0$; viene chiamato a profondità crescenti. I percorsi vengono trovati nello stesso ordine della BFS: si controlla di aver trovato un obiettivo solo se $b = 0$ (soluzioni per limiti inferiori non sono state trovate in precedenza).

Per assicurare di fallire quando anche la BFS fallirebbe, si tiene traccia dei casi in cui un limite maggiore potrebbe aiutare a trovare una soluzione. La ricerca *fallisce naturalmente* se ha esaurito tutto lo spazio di ricerca e non sono stati tagliati percorsi per limite raggiunto (in tal caso ci si può fermare restituendo $\perp$); se *raggiunta_max_profondita*, falsa al momento della chiamata, diventa vera alla fine, il limite può essere incrementato per la successiva iterata.

2.5.4 Ricerca di Soluzioni a Costo Minimo

In caso di archi con *costi* non unitari si cerca la soluzione di *minimo costo totale*. Ad esempio un robot per le consegne potrebbe considerare costo degli archi calcolati in base a distanze e/o a risorse necessarie alle azioni rappresentate dagli archi.

Gli algoritmi precedenti non garantiscono soluzioni di costo minimo: il costo non viene preso in considerazione. BFS minimizza solo il *numero* degli archi: potrebbe esistere una soluzione alternativa, i.e. un percorso *più lungo* ma di *costo inferiore*.

L'algoritmo **LOWEST COST-FIRST SEARCH** (LcFS) – detto anche **LEAST-COST SEARCH** o **UNIFORM-COST SEARCH**² – si comporta come BFS ma con la selezione dei percorsi di costo minimo. Viene implementato usando una *coda con priorità* come frontiera ordinata da *cost*.

Esempio 2.9 Si consideri il grafo in Fig. 2.3. Si espanderà sempre il percorso più a sinistra nella frontiera (costi dei percorsi indicati come pedici):

- [$\langle A \rangle_0$] frontiera iniziale
- [$\langle A, B \rangle_2, \langle A, C \rangle_3, \langle A, D \rangle_4$]
- [$\langle A, C \rangle_3, \langle A, B, E \rangle_4, \langle A, D \rangle_4, \langle A, B, F \rangle_5$]
- [$\langle A, B, E \rangle_4, \langle A, D \rangle_4, \langle A, B, F \rangle_5, \langle A, C, J \rangle_{10}$]
- [$\langle A, D \rangle_4, \langle A, B, F \rangle_5, \langle A, C, J \rangle_{10}$]
- [$\langle A, B, F \rangle_5, \langle A, D, H \rangle_8, \langle A, C, J \rangle_{10}$]
- [$\langle A, B, F, D \rangle_7, \langle A, D, H \rangle_8, \langle A, C, J \rangle_{10}$]
- [$\langle A, D, H \rangle_8, \langle A, C, J \rangle_{10}, \langle A, B, F, D, H \rangle_{11}$]
- [$\langle A, C, J \rangle_{10}, \langle A, D, H, G \rangle_{11}, \langle A, B, F, D, H \rangle_{11}$].

Dopo l'espansione di $\langle A, C, J \rangle$, si selezionerà $\langle A, D, H, G \rangle$, percorso di costo minimale da A a G .

Il fattore di ramificazione è *finito* se i *costi* sono *limitati* inferiormente da una costante positiva: ciò garantisce la soluzione ottimale, quando esiste. Il primo percorso trovato termina in un nodo-obiettivo ed è ottimale perché si procede in ordine di costo (se ne esistesse uno migliore sarebbe stato già trovato).

In mancanza di un limite inferiore allora sono possibili percorsi *infiniti*, ad esempio con nodi $n_0, n_1, n_2, \dots$ con $\forall i > 0: \langle n_{i-1}, n_i \rangle$ di costo $1/2^i$ esistono infiniti percorsi $\langle n_0, n_1, n_2, \dots, n_k \rangle$, con costo < 1 ; se $\exists \langle n_0, g \rangle$, $goal(g)$, con costo ≥ 1 , l'arco non verrebbe mai selezionato (cfr. paradossi di Zenone: **Achille e la tartaruga**).

La complessità è *esponenziale* in spazio e tempo (come per la BFS): genera tutti i percorsi con costo inferiore a quello della soluzione.

2.6 Ricerca Informata (Euristica)

2.6.1 Euristica

Tale tipo di ricerca prende in considerazione *informazioni sull'obiettivo*, nella selezione dei nodi da esplorare, attraverso l'uso di una **funzione euristica** h che

²Variante dell'algoritmo di **DIJKSTRA**, cfr. §3.4.2 in [RN20].

associa a ogni nodo n un numero reale *non-negativo*, una stima del costo minimale di un percorso da n fino a un nodo-obiettivo.

L'euristica h si dice **ammissibile** se *sottostima* il costo reale: $h(n)$ minore o uguale rispetto al costo minimale effettivo del percorso da ciascun nodo a un nodo-obiettivo. Tale informazione approssimata è spesso immediatamente disponibile trovando un *compromesso* tra efficienza del suo calcolo e accuratezza della stima. Tipicamente, risolvendo una forma semplificata del problema, si possono poi usare i costi effettivi del problema semplificato come euristica nella soluzione del problema originario.

Esempio 2.10 Problema precedente con grafo in Fig. 2.3. Considerando come *euristica* la distanza in linea retta tra nodo e obiettivo più vicino, quindi supponendo che tali distanze siano ad esempio:

	A	B	C	D	E	F	G	H	J
$h(\cdot)$	7	5	9	6	3	5	0	3	4

h risulta ammissibile e in particolare: essa è

- *esatta* per il nodo H ;
- molto *sottostimata* per il nodo B : sembra vicino al nodo-obiettivo (costo 5), ma raggiungere l'obiettivo costa molto di più;
- *ingannevole* per il nodo E : sembra vicino al nodo-obiettivo (costo 3) ma non permette di raggiungere l'obiettivo.

2.6.2 DFS Euristica e Greedy best-First Search

L'euristica h può essere *estesa* al caso dei percorsi (non vuoti):

$$h(\langle n_0, \dots, n_k \rangle) = h(n_k)$$

La **DFS EURISTICA** usa h per ordinare i *vicini* aggiunti alla pila/frontiera della DFS standard. I vicini vengono aggiunti in modo che il *migliore* vada *in cima*. Tale scelta è comunque *locale*: si esplorano i percorsi che estendono quello selezionato prima di tentarne altri. Si presentano problemi analoghi al caso della DFS: non c'è garanzia di terminazione e ritrovamento di una soluzione ottimale.

Si potrebbe considerare la **BEST-FIRST SEARCH** [RN20] che sfrutta un'euristica h da minimizzare nella scelta del prossimo cammino da espandere. In particolare, nella **GREEDY BEST-FIRST SEARCH**, GBFS, si sceglie il *percorso* della frontiera con valore di h minimo. Purtroppo potrebbe seguire cammini apparentemente promettenti (vicini all'obiettivo secondo h) che, però, potrebbero continuare a estendersi indefinitamente.

Esempio 2.11 Si consideri il grafo in Fig. 2.10. Lo scopo è trovare il percorso minimo da A a G .

- Si ignori il *costo*: i valori sui nodi sono quelli di h ;
- h : distanza euclidea da G ;
- DFS euristica e GbFS iterano indefinitamente allungando la sezione del percorso $B \rightarrow E \rightarrow F \rightarrow B \rightarrow \dots$;
- anche allorché fosse possibile identificare i cicli, un grafo simile, con infiniti nodi collegati con valore dell'euristica inferiore a 6, risulterebbe problematico per entrambi gli algoritmi.

2.6.3 Ricerca A*

L'algoritmo A* combina idee da LcFS e GbFS. Nella selezione del percorso da espandere si considerano: il costo effettivo del percorso parziale che dal nodo iniziale arriva al nodo corrente e l'euristica, ossia una stima del costo del percorso dal nodo corrente fino a un obiettivo.

Per ogni percorso $p = \langle s, \dots, n \rangle$ della frontiera, si stima il costo di un percorso che passi per p e prosegua fino a un nodo-obiettivo g :

$$f(p) = \overset{s}{\text{cost}(p)} \xrightarrow{\text{reale}} \overset{n}{\text{cost}(p)} \xrightarrow{\text{stima}} \overset{g}{\text{cost}(p) + h(p)}$$

dove $\text{cost}(p)$ rappresenta il costo effettivo di p e $h(p)$ è una stima del costo del cammino da n a g .

Si segue lo schema dell'algoritmo di ricerca generico in List. 2.1 e si rappresenta la *frontiera* con una coda con priorità, ordinata da $f(p)$. Se h è *ammissibile* allora $f(p)$ non sovrastima il costo d'un percorso completo che includa p .

A* può seguire molti percorsi ma *alla fine* ne trova uno di costo minimo. Altri possono sembrare (provvisoriamente) di costo inferiore. A* migliora le prestazioni degli algoritmi da cui origina (LcFS e GbFS).

Esempio 2.12 A* su grafo in Fig. 2.9 ed euristica.

Sequenza delle frontiere (pedice = $f(p)$)

- [$\langle A \rangle_7$] iniziale,
essendo $h(A) = 7$ e $\text{cost}(\langle \rangle) = 0$
- [$\langle A, B \rangle_7, \langle A, D \rangle_{10}, \langle A, C \rangle_{12}$]
- [$\langle A, B, E \rangle_7, \langle A, B, F \rangle_{10}, \langle A, D \rangle_{10}, \langle A, C \rangle_{12}$]
 $\langle A, B, E \rangle$ da rimuovere (E senza vicini)
- [$\langle A, B, F \rangle_{10}, \langle A, D \rangle_{10}, \langle A, C \rangle_{12}$]
tie fra i primi due percorsi ma $\langle A, B, F \rangle_{10}$ preferito perché $h(F) = 5 < 6 = h(D)$
- [$\langle A, D \rangle_{10}, \langle A, C \rangle_{12}, \langle A, B, F, D \rangle_{13}$]

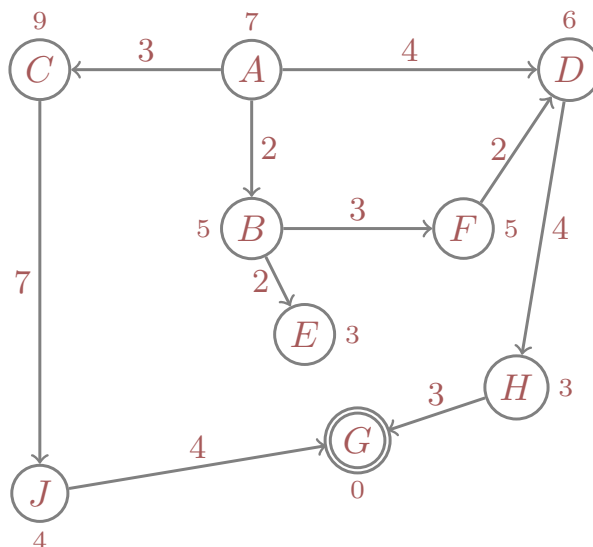


Figura 2.9: Problema con costi effettivi (sugli archi) e valori dell'euristica (sui nodi)

- $[\langle A, D, H \rangle_{11}, \langle A, C \rangle_{12}, \langle A, B, F, D \rangle_{13}]$
- $[\langle A, D, H, G \rangle_{11}, \langle A, C \rangle_{12}, \langle A, B, F, D \rangle_{13}]$

Soluzione: $\langle A, D, H, G \rangle$ di costo 11.

Esempio 2.13 Dal grafo precedente con l'aggiunta di cicli si ottiene quello in Fig.2.10. Esso risulta problematico per LcFS e GbFS. Con A*, sebbene ci si indirizzi inizialmente verso $B \rightarrow E \rightarrow F$ per via di h , il percorso diventa via via tanto costoso da far preferire presto o tardi un percorso ottimale attraverso D .

Un algoritmo è **ammissibile** se, quando esistono soluzioni, ne trova sempre una ottimale (anche in caso di uno spazio degli stati infinito).

Proposizione 2.1. A* ammissibile se:

- il fattore di ramificazione è limitato ($\leq b$)
- i costi degli archi sono maggiori di un certo $\epsilon > 0$
- h è ammissibile, i.e. non supera il costo minimale di percorsi da ciascun nodo a un obiettivo.

A* garantisce che la prima soluzione sarà ottimale, anche in presenza di cicli, ma non assicura che ciascun nodo *intermedio* selezionato dalla frontiera sia su un cammino ottimo.

Importanza dell'euristica. Il miglioramento dell'efficienza in A* è dato dall'uso di h . Sia c il costo del percorso di costo minimo. Se h è ammissibile, A* espande

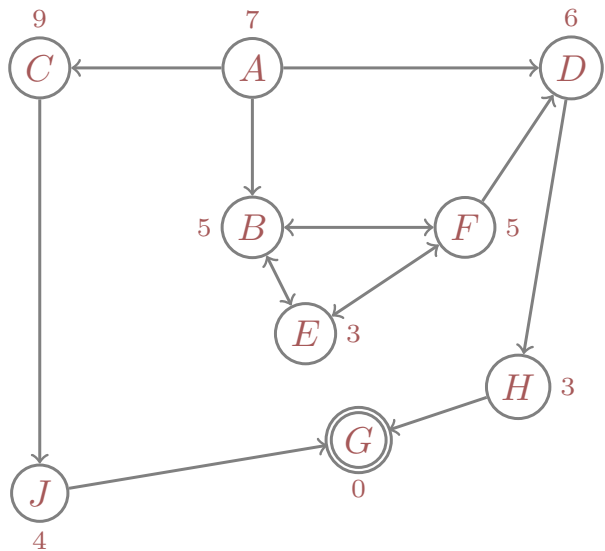


Figura 2.10: Grafo modificato con l'aggiunta di cicli

ogni percorso dal nodo di partenza nell'insieme

$$\{p \mid cost(p) + h(p) < c\}$$

più alcuni percorsi nell'insieme

$$\{p \mid cost(p) + h(p) = c\}$$

Per l'efficienza di A*, h è tanto migliore quanto più riduce la cardinalità del primo insieme.

2.6.4 Algoritmo BRANCH AND BOUND

L'idea di questo algoritmo è combinare l'efficienza in spazio delle strategie in profondità con informazione euristica. Risulta efficace quando esistono molti cammini verso un nodo-obiettivo, assume che $h(n)$ sia ammissibile.

BRANCH-AND-BOUND memorizza il percorso di costo minimo verso un nodo-obiettivo trovato e il suo costo che diventa un *limite*, *limite*: un percorso p tale che $cost(p) + h(p) \geq limite$ può essere scartato; si memorizza p completo se $cost(p) < limite$ aggiornando *limite*; si continua poi a cercare un'eventuale soluzione migliore.

Si genera così una sequenza di soluzioni via via migliori restituendo quella finale: l'algoritmo viene tipicamente implementato usando la DFS per la ricerca in profondità limitata dal valore di *limite*.

Algoritmo 2.3: BRANCH-AND-BOUND implementato con DFS

```
1 procedure DFBranchAndBound( $G, s, goal, h, limite_0$ )
```

```

2 Input
3    $G$ : grafo con nodi  $N$  e archi  $A$ 
4    $s$ : nodo di partenza
5    $goal$ : funzione di test dei nodi
6    $h$ : euristica sui nodi
7    $limite_0$ : limite iniziale ( $\infty$  se non specificato)
8 Output
9   percorso di costo minimo da  $s$  a un nodo obiettivo se esiste una
10  soluzione di costo minore di  $limite_0$ , oppure  $\perp$ 
11 Local
12   $percorso\_migliore$ : percorso o  $\perp$ 
13   $limite$ : numero reale non negativo
14
15  procedure cbsearch( $\langle n_0, \dots, n_k \rangle$ )
16    if  $cost(\langle n_0, \dots, n_k \rangle) + h(n_k) < limite$  then
17      if  $goal(n_k)$  then
18         $percorso\_migliore \leftarrow \langle n_0, \dots, n_k \rangle$ 
19         $limite \leftarrow cost(\langle n_0, \dots, n_k \rangle)$ 
20      else
21        for each  $\langle n_k, n \rangle \in A$  do
22          cbsearch( $\langle n_0, \dots, n_k, n \rangle$ )
23
24  {main}
25   $percorso\_migliore \leftarrow \perp$ 
26   $limite \leftarrow limite_0$ 
27  cbsearch( $\langle s \rangle$ )
28  return  $percorso\_migliore$ 

```

La procedura *cbsearch*, *cost-bounded search*, comunica attraverso variabili globali; inizialmente, *limite* è impostata a $limite_0$, stima per eccesso del costo d'una soluzione ottimale.

DFBranchAndBound restituirà, se esiste, una soluzione ottimale con costo inferiore a $limite_0$. Se $limite_0$ supera di poco il costo minimo, l'algoritmo non espanderà più archi di A^* . La variabile $limite_0$ serve a far eliminare percorsi di costo maggiore: trovato un percorso completo, si esplorano solo percorsi con valore di f inferiore a quello del percorso trovato, ossia esattamente i percorsi esplorati da A^* quando trova una soluzione. Quando restituisce $\perp$, se $limite_0 = \infty$ allora non ci sono soluzioni, se invece $limite_0$ è finito allora non ci sono soluzioni di costo inferiore a $limite_0$.

Combinato con ID l'algoritmo può incrementare via via il limite fino a trovare una soluzione ovvero può mostrare che non c'è soluzione.

Esempio 2.14 Considerato il grafo in Fig. 2.11, si supponga che:

- ogni arco abbia costo 1 e non ci sia informazione euristica ($\forall n : h(n) = 0$),

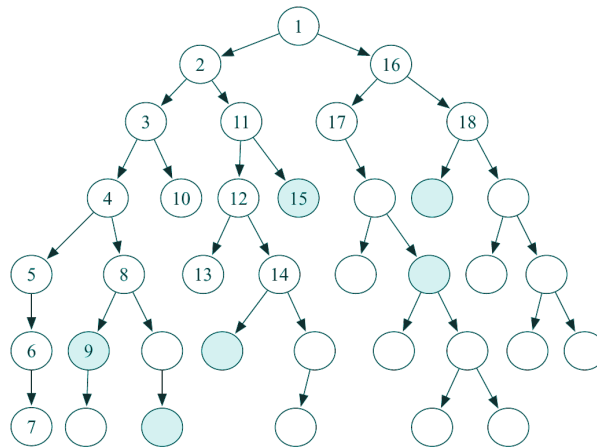


Figura 2.11: Esempio d'uso di BRANCH AND BOUND (nodi scuri = obiettivi) [PM23]

- $limite_0 = \infty$,
- si scelga sempre il nodo più a sinistra.

Si noti l'ordine di test dei nodi (via *goal()*): i nodi non numerati non sono testati. Il sotto-albero del nodo 5 non ha nodi-obiettivo e viene esplorato tutto o fino a profondità $limite_0$, se finito. Il nodo 9 è un nodo-obiettivo, con percorso di costo 5, che diventa il nuovo limite. Anche il nodo 15 è un nodo-obiettivo, con percorso di costo 3 quindi si aggiorna il limite. Quando non ci sono altri nodi-obiettivo si restituisce il percorso fino al nodo 15 ottimale; l'altro, di pari costo, viene scartato: non si controllano i figli del nodo 18.

Si potrebbe usare un'euristica per tagliare parti dello spazio come in A*.

2.6.5 Progettazione della Funzione Euristica

Un'euristica ammissibile è una funzione non negativa h definita sui nodi, tale che dato un nodo n , $h(n)$ non è mai superiore al costo reale del percorso ottimale da n a un nodo-obiettivo.

Una procedura standard per definire un'euristica può essere descritta come segue:

- Si trovano soluzioni di un problema più semplice, ossia con *meno vincoli*, quindi spesso molto *più facile* da risolvere; una sua soluzione ottimale avrà un costo pari o inferiore a una soluzione del problema originario (ognuna lo sarà anche per la versione semplificata).
- Si considerino problemi di ricerca su mappe semplificate rappresentati da grafi, in cui si sia *vincolati* a muoversi lungo i dati archi (i.e. le strade esistenti): supponendo che il loro costo sia proporzionale alla lunghezza dei collegamenti stradali, un'euristica ammissibile è costituita dalla *distanza eu-*

clidea tra punti/nodi (problema semplificato, i.e. privo dei suddetti vincoli, ci si sposta in linea d'aria).

Esempio 2.15 (robot-consegne) In un problema più complesso, si considerino:

- *stati* comprendenti le consegne da fare;
- il *costo* dato dalla distanza totale per tutte le consegne;
- un'*euristica* possibile pari al massimo tra:
 1. la distanza massima per consegne ancora da fare non caricate: per ognuna, la distanza dalla sua posizione e, da qui, fino alla destinazione;
 2. la distanza dalla destinazione più lontana per le consegne trasportate.

Tale massimo non sovrastima il costo: è la soluzione per un problema semplificato che non considera muri e tutte le altre consegne da fare tranne la più difficile; è appropriato: vanno consegnate quelle trasportate, passate a prendere quelle ancora non caricate e portate a destinazione.

Se il robot può trasportare una sola consegna, un'*euristica* dovrebbe sommare le distanze per il trasporto di ognuna più la distanza dalla più vicina: non è detto che sarà effettuata per prima (ma serve a garantire l'ammissibilità).

Esempio 2.16 (navigatore) Per la minimizzazione del *tempo*:

- possibile *euristica*: distanza in linea retta dalla locazione corrente al nodo-obiettivo divisa per la velocità massima, assumendo che ci si possa dirigere a destinazione alla massima velocità;
- *euristica* più sofisticata: date velocità massime diverse per autostrade e strade locali, si sceglie il massimo tra:
 1. minimo tempo stimato per andare a destinazione su strade locali lente
 2. minimo tempo utile a raggiungere un'autostrada da una strada locale, per poi raggiungere un luogo vicino alla destinazione e arrivarci da strada locale.

Un problema in forma semplificata può essere risolto attraverso una ricerca più semplice rispetto al problema originario: ricerca ripetuta anche più volte e per tutti i nodi.

Conviene memorizzare (in una memoria *cache*) tali risultati in un **DB di pattern** che associ ai nodi del problema semplificato il valore dell'*euristica*.

Nel problema semplificato, spesso si hanno meno nodi: più nodi originari sono mappati su uno *stesso* nodo di quello semplificato come fosse una *classe*

d'equivalenza.

2.7 Potatura dello Spazio di Ricerca

Si possono migliorare gli algoritmi di ricerca considerando i diversi percorsi che possono attraversare un nodo. Non tutti sono utili a risolvere il problema per cui gli altri possono essere subito eliminati.

Essenzialmente vi sono due tipi di **pruning** (*potatura*) con relative strategie:

- *potare i cicli* per trovare percorsi dal costo minimo;
- *potare cammini molteplici*: per ogni nodo si considererà un solo percorso che lo attraversi, eliminando tutti gli altri.

2.7.1 Potatura dei Cicli

Alcuni dei metodi di ricerca visti in precedenza potrebbero rimanere intrappolati in cicli senza possibilità di uscita anche nel caso di grafi finiti.

Un esempio di grafo contenente cicli è quello in Fig. 2.8: un ciclo contenuto in più cammini è il seguente: $B \rightarrow F \rightarrow D \rightarrow A \rightarrow B \dots$. Per avere la garanzia di trovare una soluzione in caso di *grafo finito* non andranno presi in considerazione vicini già presenti nel percorso corrente.

Nella **potatura dei cicli** (*cycle / loop pruning*, CP), si prevede un controllo aggiuntivo preventivo: dato un nodo da aggiungere, va testata l'occorrenza nel percorso.

Nello schema di algoritmo di ricerca List. 2.1 si aggiungono alla frontiera *solo* percorsi $\langle n_0, \dots, n_k, n \rangle$, tali che $n \notin \{n_0, \dots, n_k\}$, scartando il percorso selezionato; in alternativa si può effettuare il controllo *dopo* la selezione del nodo.

La complessità dei controlli aggiuntivi per la verifica della presenza di un nodo nel percorso corrente dipende dal metodo di ricerca adottato:

- *costante* per metodi (DF) che memorizzano un *unico percorso* (in una lista o insieme); si può usare una funzione *hash* oppure si può associare un *bit* a ogni nodo (flag booleano), che sarà *acceso* quando viene aggiunto a un percorso e viene *spento* in caso di backtracking; per cui basterà non espandere nodi con il bit acceso; tale metodo funziona perché si lavora su un solo percorso;
- *lineare* nella lunghezza del percorso corrente per metodi che gestiscono più percorsi (esponenziali in spazio): serve una ricerca per evitare di aggiungere al percorso parziale un nodo già presente.

2.7.2 Potatura di Percorsi Multipli

Spesso più di un percorso porta a uno stesso nodo. Occorre quindi *eliminare dalla frontiera ogni percorso che porti a un nodo per il quale ne esista già un altro* (di costo inferiore).

Una strategia di **multiple-path pruning** (MPP) può essere implementata gestendo una lista di nodi terminali di percorsi già esplorati, detta **closed list** o *explored set*: inizialmente essa è vuota poi, selezionato un percorso $\langle n_0, \dots, n_k \rangle$ della frontiera, se n_k è già nella lista esso può essere scartato, altrimenti si aggiunge n_k alla lista e si prosegue. Si veda l'algoritmo di Figura 3.16 in [PM23].

Si noti che questa strategia non garantisce che non venga eliminato un percorso di costo minimo. Per garantire di preservare le soluzioni ottimali vi sono alcune alternative:

- assicurare che il *primo* percorso trovato per un dato nodo sia ottimale quindi gli altri si possono *eliminare*;
- se nella frontiera si dovesse inserire un percorso $p = \langle s, \dots, n, \dots, m \rangle$, ma fosse già presente $p' = \langle s, \dots, n \rangle$ meno costoso della parte fino a n in p , si può: *eliminare* p oppure *oppure sostituire* in p tale parte con p' .

In LcFS, il primo percorso verso un dato nodo (al momento della selezione dalla frontiera) è quello di costo minimo. Pertanto potare altri percorsi non elimina un cammino a costo minimo per il nodo. Ciò assicura di poter trovare una soluzione ottimale.

A* non garantisce che quando si seleziona un percorso verso un nodo per la prima volta, questo sia quello di costo minimo: il teorema sull'ammissibilità lo garantisce *solo* per i percorsi verso nodi-obiettivo; per quelli verso altri nodi dipende dalle *proprietà dell'euristica*.

Un'**euristica consistente** h (non negativa) soddisfa il vincolo:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

per ogni coppia di nodi n e n' .

Se $h(g) = 0$ per ogni nodo-obiettivo g , allora h sarà consistente e non sovrastimerà mai il costo dei percorsi da un nodo verso un obiettivo.

La consistenza è garantita se h soddisfa la **restrizione di monotonicità**:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

per ogni arco $\langle n, n' \rangle$. Ciò è più facile da testare: dipende solo dagli archi, non dalle coppie di nodi considerate.

Considerato un grafo in Fig. 2.12, per la *consistenza*, il costo stimato del percorso da n al nodo-obiettivo g non dev'essere maggiore della stima di quello che obbliga al passaggio da n' . Ad esempio ciò vale con la distanza euclidea e con euristiche indotte dalle soluzioni di problemi semplificati.

La *restrizione monotona* richiede che i valori di f dei percorsi selezionati dalla frontiera siano monotonicamente non decrescenti.

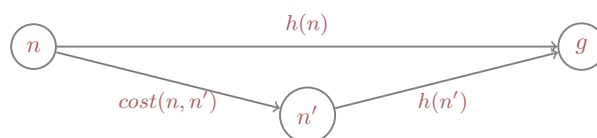


Figura 2.12: Consistenza e disuguaglianza triangolare

Tabella 2.1: Consuntivo dei diversi algoritmi: si considera la complessità in spazio, con d profondità e b limite superiore per il fattore di ramificazione.

strategia	selezione dalla frontiera	garanzia soluzione	complessità (spazio)
BFS	primo nodo aggiunto	con meno archi	$O(b^d)$
DFS	ultimo nodo aggiunto	no	$O(bd)$
ID	n/a	con meno archi	$O(bd)$
GBFS	$h(p)$ minimale	no	$O(b^d)$
LcFS	$cost(p)$ minimale	costo minimo	$O(b^d)$
A*	$cost(p) + h(p)$ minimale	costo minimo	$O(b^d)$
DF B&B	n/a	costo minimo	$O(bd)$
IDA*	n/a	costo minimo	$O(bd)$

Proposizione 2.2. Data un'euristica consistente, MPP non impedisce ad A* di trovare una soluzione ottimale.

Quindi, nelle condizioni della proposizione sull'ammissibilità di A* che garantiscono il ritrovamento di una soluzione ottimale, se l'euristica è consistente, anche A* con MPP la potrà trovare.

Nella pratica, A* include MPP per default. Andrebbe evidenziato esplicitamente che si usa A* senza potatura; compito del progettista è quello di definire un'euristica consistente in modo da permettere il ritrovamento di percorsi ottimali.

Confronto fra MPP e CP.

- MPP è *più generale* di CP: un ciclo può essere considerato come un altro percorso verso un dato nodo, destinato a essere potato;
- MPP può essere implementato in modo da richiedere un tempo costante;
- MPP è preferibile con metodi *in ampiezza*: virtualmente tutti i nodi considerati vanno memorizzati;
- CP è preferibile con le strategie *in profondità*, ad esempio DFS con BRANCH-AND-BOUND; con MPP+DFS c'è il rischio di crescita esponenziale per la lista chiusa.

2.7.3 Consuntivo sulle Strategie di Ricerca

Un consuntivo schematico sulle diverse strategie di ricerca e la relativa complessità in spazio è disponibile in Tab. 2.1.

Un **algoritmo** di ricerca **completo** garantisce il ritrovamento di una soluzione quando esiste. Le strategie che trovano percorsi con numero archi o costo minimale sono complete: nel *caso pessimo* serve un tempo esponenziale nel numero di archi dei percorsi esplorati. L'esistenza di algoritmi completi con minore complessità dipende dalla risposta alla domanda

$$P \neq NP?$$

Per algoritmi che non garantiscono la terminazione la complessità risulta non limitata.

2.8 Strategie di Ricerca più Sofisticata

Sono possibili alcune estensioni delle strategie di ricerca presentate:

- dato che decidere la direzione della ricerca impatta sull'efficienza, si può pensare a una ricerca retrograda per soluzioni ottimali da qualunque nodo;
- per determinare una buona euristica si può usare la *programmazione dinamica*: metodi basati sulla *decomposizione* del problema in un numero di problemi equivalenti ma di minore complessità.

2.8.1 Direzione della Ricerca

Nella **ricerca in avanti** (*forward search*) si comincia da un nodo di partenza e si prosegue fino a raggiungere i nodi-obiettivo. Nella **ricerca retrograda** (*backward search*) si parte da un nodo-obiettivo e si usa il *grafo inverso* alla ricerca del nodo di partenza; i vicini del nodo-obiettivo considerati per inizializzare la frontiera sono tutti gli altri nodi-obiettivo: $\{n : goal(n)\}$.

La direzione di ricerca è *indifferente* quando: 1) il numero di nodi-obiettivo *finito*; 2) per ogni nodo n si possono generare i vicini nel **grafo inverso** $\{n' : \langle n', n \rangle \in A\}$.

Le dimensioni dello spazio di ricerca sono date da b^d , dove b è il fattore di ramificazione e d la lunghezza del cammino. Riducendo questi parametri si abbassa la complessità aumentando l'efficienza. È possibile che vi sia una differenza tra i fattori di ramificazione uscente ed entrante; in tal caso un *principio generale* per la scelta della direzione della ricerca è quello di considerare la direzione per cui b risulta inferiore.

2.8.2 Ricerca Bidirezionale

Nella **ricerca bidirezionale** si riduce il tempo di ricerca procedendo in entrambe le direzioni: quando le due frontiere *si intersecano*, si deve ricostruire un singolo cammino dal nodo di partenza al nodo-obiettivo.

Problema: come assicurare che le due frontiere s'incontrino?

- caso di DFS: è difficile date le ridotte dimensioni delle frontiere;
- caso di BFS: è garantito.

Combinando BFS e DFS in direzioni opposte, si garantirebbe l'intersezione, ma quale direzione per ciascun algoritmo? Il *fattore* per tale scelta è per la BFS il costo del mantenimento *della* frontiera (tempo proporzionale a b^k) mentre per la DFS è il costo della ricerca *nella* frontiera per trovare un'intersezione; se si tratta di una ricerca simmetrica *bidirezionale* si è nell'ordine di $2b^{k/2}$, con risparmi (esponenziali) in tempo, ma la complessità rimane esponenziale.

2.8.3 Ricerca in una Gerarchia di Astrazioni

L'idea è quella di astrarre la definizione del problema, rimuovendo dettagli: una soluzione parziale del problema potrà essere ottenuta da una soluzione per la versione astratta. Ad esempio per un robot passare da una stanza a un'altra richiede più modi di girare ma questi potrebbero essere omessi nella forma astratta. Con l'astrazione si punta a risolvere un problema a grana grossa, lasciando da risolvere problemi più specifici e semplici.

Diverse sono le modalità di *decomposizione / astrazione* di problemi. Tutte le strategie di ricerca presentate sono utilizzabili ma le specifiche decomposizioni / astrazioni utili non risultano facili da individuare.

Esempio 2.17 (ricerca a isole) Si consideri una ricerca fra i diversi piani di un edificio. Trovata la soluzione a livello di isola si passa a risolvere ricorsivamente i sotto-problemi in modo analogo:

- informazioni sulle soluzioni trovate a livelli più bassi possono servire a quelle per livelli più alti;
- ai livelli più alti si può usare tale informazione per riformulare un piano di ricerca;
- il processo tipicamente non garantisce soluzioni ottimali perché considera solo alcune delle decomposizioni possibili.

2.8.4 Uso della Programmazione Dinamica

La **programmazione dinamica** (PD) fornisce un metodo generale di ottimizzazione basato sulla memorizzazione di *soluzioni parziali*: una soluzione per un

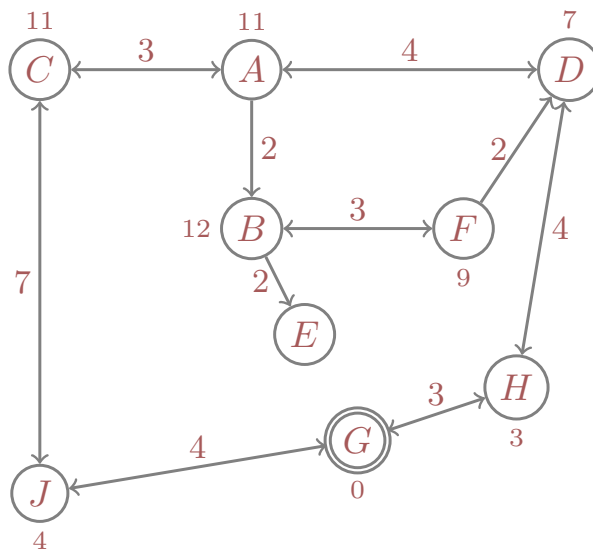


Figura 2.13: Grafo con costi ricavati con la PD

problema più semplice già prodotta (e memorizzata) dev'essere solo ritrovata e non ricalcolata.

Si può ricorrere alla PD per la costruzione offline di un'euristica perfetta $cost_to_goal()$, che fornisca il costo *esatto* di un cammino completo di costo minimo su grafi finiti:

$$cost_to_goal(n) = \begin{cases} 0 & \text{se } goal(n) \\ \min_{\langle n,m \rangle \in A} [cost(\langle n,m \rangle) + cost_to_goal(m)] & \text{altrimenti} \end{cases}$$

calcolato usando LcFS + MPP sul *grafo inverso a partire dai nodi-obiettivo*: si cerca il cammino di costo minimo da ogni nodo verso i nodi-obiettivo e si conserva il valore di $cost_to_goal$ per ogni nodo.

Esempio 2.18 (uso della PD) Dato il grafo in Fig. 2.3, risulta:

- $cost_to_goal(G) = 0$, essendo il nodo-obiettivo;
- $cost_to_goal(H) = 3$ e $cost_to_goal(J) = 4$;
- $cost_to_goal(C) = 11$ e $cost_to_goal(D) = 7$...

Si ottengono i valori dell'euristica indicati in Fig. 2.13.

Si noti come nessun valore sia previsto per E .

Una **policy** è una politica da seguire nella specifica dell'arco da considerare per ogni nodo; essa è **ottimale** se i suoi costi non sono mai superiori ai costi calcolati usando altre policy.

- $cost_to_goal$ viene calcolata *offline* e usata nella costruzione di una policy: da n si dovrebbe andare al vicino m che minimizza $cost(\langle n,m \rangle) + cost_to_goal(m)$;

questa policy porta sempre a un obiettivo con un camminino di costo minimo partendo da qualsiasi nodo;

- alternative: 1) memorizzare il vicino per tutti i nodi *offline*, sfruttando le associazioni tra nodi nelle decisioni online sulle azioni oppure 2) fornire *cost_to_goal* pre-calcolata e calcolare i vicini online.

Riguardo la complessità, si può dire in breve che:

- PD è *lineare* in tempo e spazio rispetto alle dimensioni del grafo nella costruzione di *cost_to_goal*, tipicamente *esponenziali* nella lunghezza del percorso;
- Data *cost_to_goal*, determinare l'arco migliore richiede tempo *costante* rispetto alla grandezza del grafo, essendoci un numero limitato di vicini per nodo.

La PD può servire a costruire euristiche per A* e BRANCH-AND-BOUND: semplificando il problema fino a ottenere spazi di ricerca ridotti e trovando in tali spazi soluzioni di lunghezza ottimale; ciò definisce un *DB di pattern* usato nell'euristica per il problema originale.

La PD è utile quando i nodi-obiettivo possono essere elencati *esplicitamente* (senza dover usare *goal()*), la soluzione cercata è un cammino di *costo minimo*, il grafo ridotto è *finito*, con memoria sufficiente per contenere la tabella, l'obiettivo non cambia (frequentemente). La policy può essere riusata per i diversi nodi-obiettivo: si ammortizza il costo per produrre la tabella su diverse istanze dello stesso problema; tuttavia essa va calcolata per ogni diverso nodo-obiettivo.

Approfondimenti e Link

Si veda il Cap. 3 in [RN20].

Esercitazioni pratiche: codice contenuto in *Graph search*: <http://aispace.org/search/>

- State space search: https://en.wikipedia.org/wiki/State_space_search;
- IDA*: https://en.wikipedia.org/wiki/Iterative_deepening_A*